

9. HTTP2.0/3.0

1. HTTP 2.0

这篇文章我们来聊一聊 HTTP 2.0，以及 HTTP 2.0 它在 HTTP 1.1 的基础上做了哪些改变，以及 HTTP 2.0 都有哪些特征，那么废话不多说，下面开始本篇文章。

1.1. 最初的HTTP

HTTP 刚刚诞生之初只用于 web 端的内容获取，一般就是用于页面访问，那个时候的页面内容还不如现在这样丰富，交互场景也不是很多，也没有庞大繁杂的 CSS、JS，页面加载速度非常快。但是随着 web 2.0 的出现以及更多的内容被展示、更精美的排版、更多的用户交互场景一起出现，导致页面的内容越来越大，使得页面加载速度越来越慢。

1.2. HTTP的瓶颈

影响一个 HTTP 网络请求的因素主要有两个：**带宽和延迟**。

首先来说一下带宽，如果我们还停留在拨号上网阶段的话，带宽很容易出现瓶颈，因为单位时间内传输的数据量很小。但是现在随着光纤等通信技术的不断发展，10Mbps、100Mbps、甚至 1000 Mbps 进入了每个家庭，我们不用再担心带宽成为网络请求的瓶颈了，那么剩下的就只剩下延迟了。

延迟主要有下面三个方面：

- 浏览器阻塞（HOL blocking）：浏览器会因为一些原因阻塞请求。浏览器对于同一个域名，同时只能有 6 个连接（这个根据浏览器内核不同可能会有所差异），超过浏览器最大连接数限制，后续请求就会被阻塞。
- DNS 查询（DNS Lookup）：浏览器需要知道目标服务器的 IP 才能建立连接。将域名解析为 IP 的这个系统就是 DNS。这个通常可以利用 DNS 缓存结果来达到减少这个时间的目的。
- 建立连接（Initial connection）：HTTP 是基于 TCP 协议的，浏览器最快也要在第三次握手时才能捎带 HTTP 请求报文，达到真正的建立连接，但是这些连接无法复用会导致每次请求都经历三次握手和慢启动。三次握手在高延迟的场景下影响较明显，慢启动则对文件类大请求影响较大。

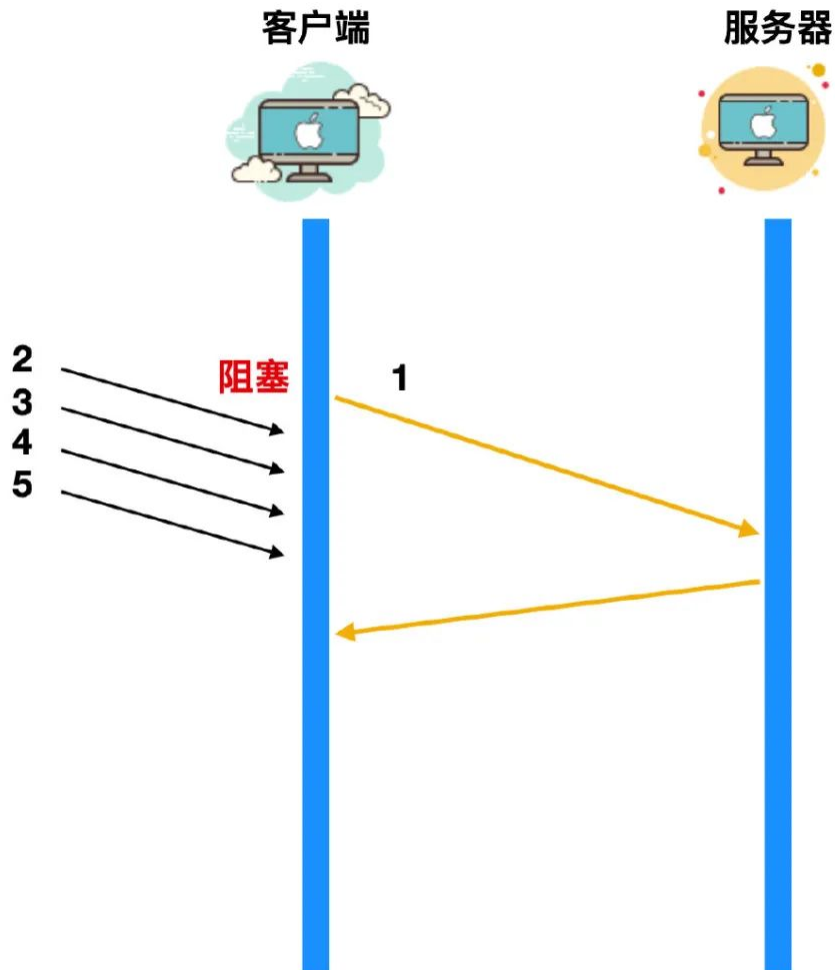
HTTP 1.0 有一个被抱怨最多的是**连接无法复用**，当每次有新的请求时都会重新经历一次三次握手和四次挥手过程，并且连接的建立和释放需要耗费大量的服务器资源，在请求少的页面还尚能应对，不过随着请求的不断增多，HTTP 1.0 越来越难顶。

不过，HTTP 1.0 协议头里可以设置 Connection:Keep-Alive。在 header 里设置 Keep-Alive 可以在一定时间内复用连接，具体复用时间的长短可以由服务器控制，一般在 15s 左右。到 HTTP 1.1 之后

Connection 的默认值就是Keep-Alive，如果要关闭连接复用需要显式的设置 Connection:Close。

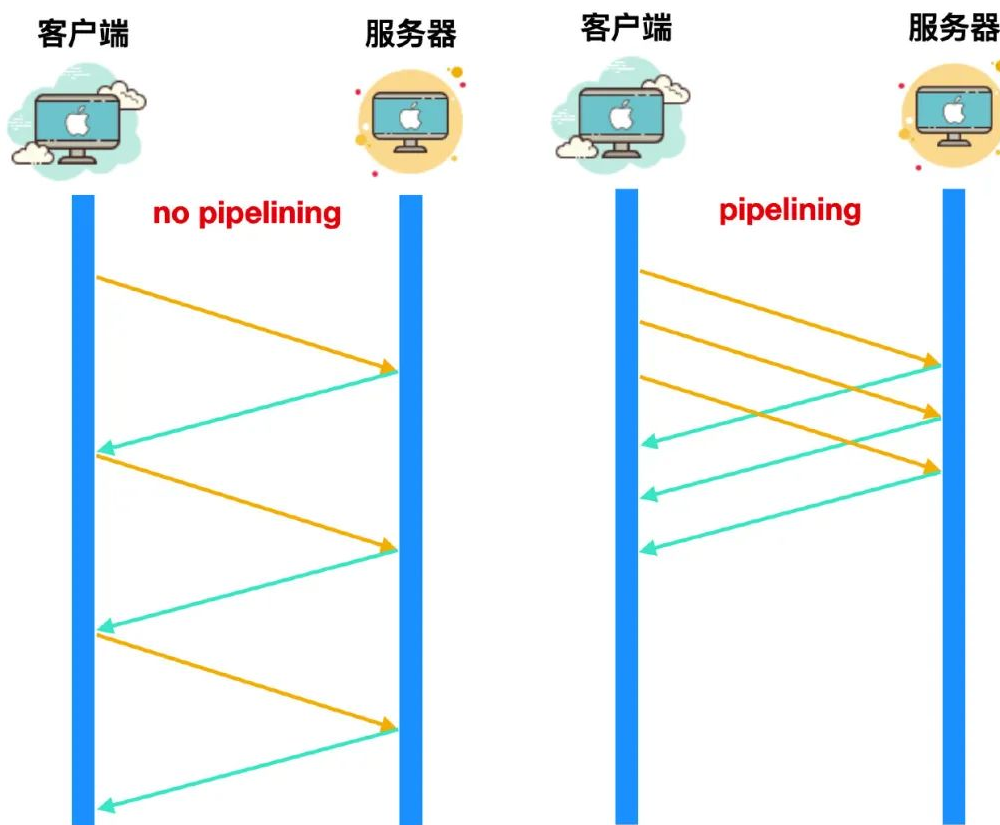
HTTP 还有一个被抱怨最多的问题就是它的**队头阻塞(head of blocking)**，队头阻塞问题会导致带宽无法充分利用，导致后续的请求被阻塞。

假如有五个请求被同时发出，如果第一个请求没有处理完成，就会导致后续的请求也无法得到处理，如下图所示：



如果第一个请求没有被处理，那么 2 3 4 5 这四个请求会直接阻塞在客户端，等到请求 1 被处理完毕后，才能逐个发出。网络通畅的时候性能影响不大，不过一旦请求 1 因为某些原因没有抵达服务器，或者请求因为网络阻塞没有及时返回，影响的就是所有后续请求，导致后续请求无限阻塞下去，问题就变得比较严重了。

不过在 HTTP 1.1 中，也提出了**流水线(pipelining)**的设计，pipelining 就被用来解决队头阻塞的问题，如下图所示：



虽然这种流水线的设计乍一看像是能够解决阻塞问题，因为右图中这三个请求没有等到响应到达后再进行发送，而是直接依次发送，但是实际上，并不是那么回事。

pipelining 并不是救世主，它也存在不少缺陷：

- 因为只有幂等的请求比如 GET、HEAD 才能使用 pipelining，非幂等请求比如 POST 则不能使用，因为请求之间可能存在先后依赖关系。
- 其实队头阻塞问题并没有完全解决，因为服务器返回的响应还是要依次返回，也就是返回的请求时 FIFO – 先发先回。
- 绝大多数 HTTP 代理服务器不支持 pipelining。
- 和不支持 pipelining 的老服务器协商有问题。

正是因为有这么多的问题，各大浏览器厂商要么是根本就不支持 pipelining，要么就是默认关掉了 pipelining 机制，而且启用的条件十分苛刻。

1.3. SPDY

虽然 HTTP1.0 和 HTTP 1.1 存在这么多问题，业界也是想出了各种优化手段，但是这些手段怎么说呢，都是治标不治本，直到 2020 年 Google 提出了 **SPDY** 的方案，大家才开始从正面看待和解决老版本 HTTP 协议本身的问题，这也直接加速了 HTTP 2.0 的诞生。

我们先来聊一下 SPDY 是什么，它都有哪些特点？

1.3.1. 认识SPDY

SPDY 的目标在于解决 HTTP 的缺陷，即延迟和安全性。我们上面一直在讨论延迟，至于安全性，虽然我们上面没有具体聊，不过 HTTP 的明文传输确是个问题。如果以降低延迟为目标，应用层的 HTTP 和传输层的 TCP 都是都有调整的空间，不过 TCP 作为更底层协议存在已达数十年之久，其实现已深植全球的网络基础设施当中，如果要动必然伤经动骨，业界响应度必然不高，所以 SPDY 的手术刀对准的是 HTTP。

- 降低延迟，客户端的单连接单请求，服务端的 FIFO 响应队列都是延迟的大头。
- HTTP 最初设计都是客户端发起请求，然后服务端进行响应，服务端无法主动发送内容到客户端。
- 压缩 HTTP header，HTTP 1.x 的 header 越来越膨胀，cookie 和 user agent 很容易让 header 的 size 增至1kb 大小甚至更多。而且由于 HTTP 的无状态特性，header 必须每次请求都重复携带，很浪费流量。

为了增加解决这些问题的可行性，聪明的 Google 一开始就避开了从传输层动手，而且打算利用开源社区的力量以提高扩散的力度，对于协议使用者来说，也只需要在请求的 header 里设置 user agent，然后在服务端做好支持即可，极大的降低了部署的难度。SPDY 的设计如：



可以看到，SPDY 位于 HTTP 之下，SSL 之上，这样可以轻松的兼容老版本的 HTTP 协议，SPDY 的功能分为基础功能和高级功能两部分，基础功能是默认启用的，高级功能需要手动启用。

1.3.2. SPDY基础功能

- **多路复用(multiplexing)**，多路复用通过多个请求共用一个连接的方式，降低了 TCP 连接建立和释放的开销，同时提高了带宽的利用率。
- **请求优先级(request prioritization)**，多路复用带来的一个问题是，在共享连接的基础上会存在一些关键请求被阻塞，SPDY 允许给每个请求设置优先级，这样重要的请求就会优先得到响应。
- **header 压缩**，前面提到的 HTTP 1.x 的 header 很多时候都是重复而且多余的。选择合适的压缩算

法可以减小包的大小和数量。SPDY 对 header 的压缩率可以达到 80% 以上。

1.3.3. SPDY高级功能

- 服务端推送，HTTP 只能由客户端发送，服务器只能被动发送响应。不过在开启服务端推送后，服务端通过 **X-Associated-Content header** 会告知服务器会有新的内容被推送过来，
- 服务端暗示，和服务端推送所不同的是，服务端暗示不会推送内容，只是告诉客户端有新的内容产生，，内容的下载还是需要客户端主动发起请求。服务端暗示通过 X-Subresources header 来通知，一般应用场景是客户端需要先查询服务端状态，然后再下载资源，可以节约一次查询请求。

1.4. 初探HTTP 2.0

HTTP 2.0 也被写作 HTTP/2，它是超文本传输协议的 2.0 版本，因为 SPDY 的流行让 IETF 看到了优化后的效果，以及可以通过修改协议层来优化 HTTP，所以 IETF 开始决定正式考虑制定 HTTP 2.0 的计划，而且，SPDY的部分设计人员也被邀请参与了 HTTP 2.0 的设计。

HTTP2.0 在设计之初就与 SPDY 的设计目的和出发点不同，SPDY 更像是 Google 自家的一个产品，相当于自家的一个玩具，你怎么玩儿都行，而 HTTP 2.0 在设计之初就是为了普适性的这个目的，所以，一开始任何的设计都会关系到以后的维护问题，如果有什么瑕疵或者不足的地方可能会影响巨大，所以考虑的问题角度要非常严谨和慎重。

HTTP 2.0 在设计之初就有一些重要的前提：

- 客户端向服务器发送请求的这种基本模型不会改变。
- 原有的协议头不会改变，使用 http:// 和 https:// 的服务和应用不会做任何修改，不会有 http2://。
- 使用 HTTP 1.x 的客户端和服务器可以平滑升级到 HTTP 2.0 上。
- 不识别 HTTP 2.0 的代理服务器可以将请求降级到 HTTP 1.x。

客户端在和服务器确定是使用 HTTP1.x 还是 HTTP 2.0 之前，需要先确定对方是否支持 HTTP 2.0，所以这里必须要先进行协商，也就是客户端询问服务器，这样一来一回就多了一个 RTT 的延迟。我们对 HTTP 1.x 的修改就是为了降低延迟，现在又多了一个 RTT，这样显然是无法接受的。Google 制定 SPDY 协议的时候也遇到了这个问题，他们采取的做法是强制协商在 SSL 层完成，还因此制定了一个 TLS 的拓展，叫做 **NPN(Next Protocol Negotiation)**。虽然 HTTP 2.0 也采用了相同的方式，不过经过讨论后，最终 HTTP 2.0 没有强制要走 SSL 层，HTTP 2.0 没有使用 NPN，却制定了一个 TLS 的拓展叫做 **ALPN(Application Layer Protocol Negotiation)**，现在，SPDY 也打算迁移到 ALPN 了。

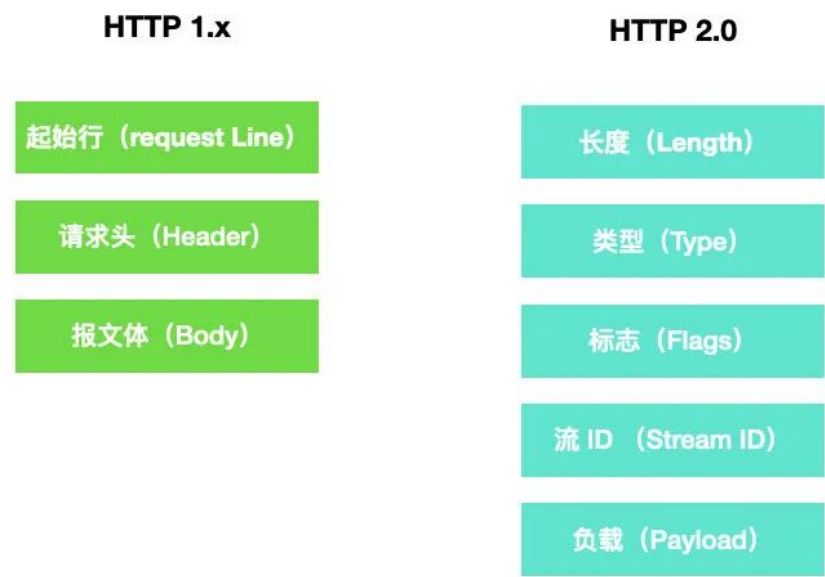
1.5. HTTP 2.0的主要变化

HTTP 2.0 自从设计到诞生以来，发生了很多变化，不过对于开发人员和厂商来说，影响比较大的就几点：

1.5.1. 二进制格式

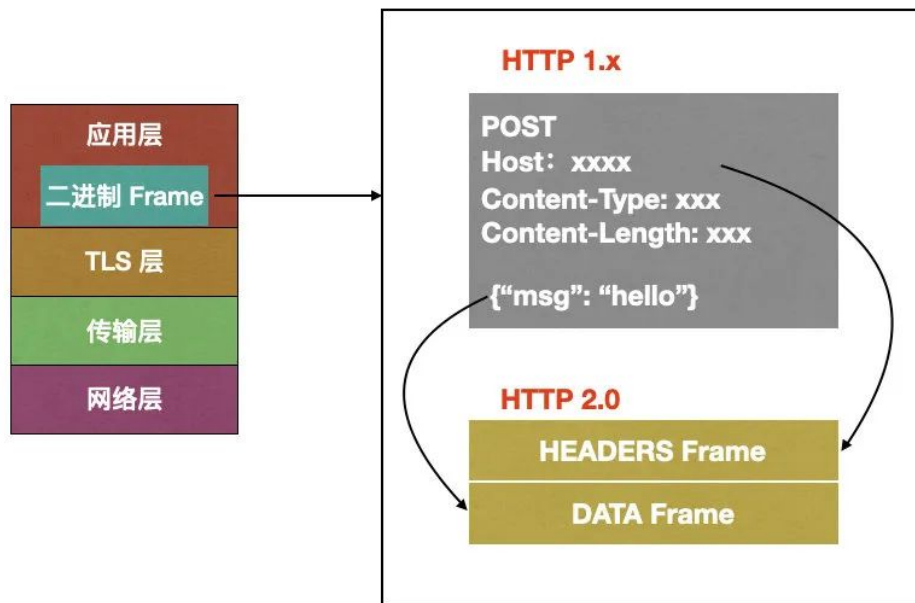
HTTP 1.x 的诞生使用的是明文协议，它的格式主要由三部分构成：请求行(request line)、请求头(header) 和报文体(body)，要识别这三部分必须要做协议解析，而协议解析是基于文本的，基于文本的解析存在多样性的缺陷，而二进制格式只能识别 0 和 1，比较固定，基于这种考量，HTTP 2.0 决定采用二进制格式，实现方便而且健壮性强。

下面这幅图很好的诠释了 HTTP1.x 和 HTTP 2.0 使用的不同报文格式。



在 HTTP 2.0 报文中，length 定义了整个 frame 的开始到结束，type 定了 frame 的类型，一种有十种，flags 定义了一些重要的参数，stream id 用作流控制，剩下的 payload 就是 request 的正文。

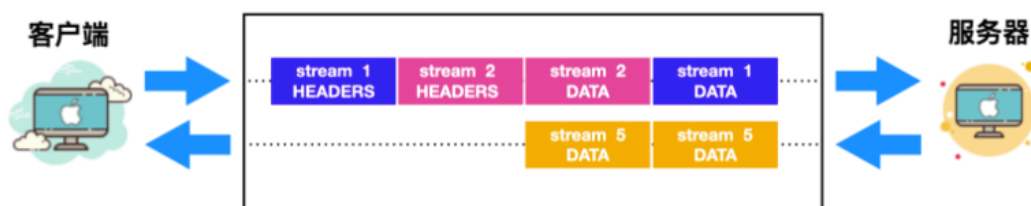
虽然 HTTP 2.0 报文格式看上去和 HTTP 1.x 的完全不同，但是实际上 HTTP 2.0 并没有改变 HTTP 1.x 的语义，它只是在 HTTP 1.x 的基础上封装了一层，如下图所示：



从上图可以看到，HTTP 1.x 中的请求行、请求头被 HTTP 2.0 封装成为了 HEADERS Frame，而 HTTP 1.x 中的报文体被 HTTP 2.0 封装成为了 Data Frame。调试的时候浏览器会把 HTTP 2.0 的 frame 自动还原成 HTTP 1.x 的格式。

1.5.2. 连接共享

我们上面聊到，HTTP 1.x 并没有真正意义上的解决连接复用问题，所以 HTTP 2.0 要解决的一大难题就是**连接共享(MultiPlexing)**，连接共享意味着客户端与服务器之间也只需要一个连接即可，这样即使来自很多流的数据包也能够混合在一起通过同样连接传输，再根据不同帧首部的 stream id 标识符重新连接将不同的数据流进行组装。



什么是 stream?

stream 是连接中的一个虚拟信道，可以承载双向消息传输。每个流有唯一整数标识符。为了防止两端 stream id 冲突，客户端发起的流具有奇数 id，服务器端发起的流具有偶数 id。

我们上面提到 HTTP 1.x 没有真正解决连接共享还有一个主要的因素就是**无法对不同的请求设置优先级**，这样会导致关键请求被阻塞。而 HTTP 2.0 你可以对不同的 stream 设置不同的优先级，stream 之间也可以设置依赖，依赖和优先级都可以动态调整，这样就会解决关键请求被阻塞的问题。

1.5.3. 头部压缩

上面还聊到了 HTTP1.x 中的 header 由于 cookie 和 user agent 不存在记忆性，这样导致每次都要带着这些头重新发送请求，HTTP 2.0 使用 encoder 来减少传输的 header 大小，通信双方会各自缓存一份 header 字段表，这样能够避免重复传输 header，也能够减小传输的大小。HTTP 2.0 采用的是 HPACK 压缩算法。

这种压缩算法的主要思想可以参考官方文档 <https://httpwg.org/specs/rfc7541.html>

1.5.4. 服务端推送

服务端推送(Server Push) 我们上面也已经聊过，HTTP 2.0 能够以**推**的方式将客户端的内容预先发送出去，正因为没有发起请求，建立连接等操作，所以静态资源通过服务端推送的方式可以极大地提升速度。服务端推送还有一个更大的优势：**缓存**，缓存也能够不同页面之间共享缓存资源。

需要注意下面几个点：

- 1、推送遵循同源策略；
- 2、这种服务端的推送是基于客户端的请求响应来确定的。

当服务端需要主动推送某个资源时，便会发送一个 Frame Type 为 PUSH_PROMISE 的 frame，里面带了 PUSH 需要新建的 stream id。意思是告诉客户端：接下来我要用这个 id 向你发送东西，客户端准备好接着。客户端解析 frame 时，发现它是一个 PUSH_PROMISE 类型，便会准备接收服务端要推送的流。

1.6. HTTP 2.0的缺陷

HTTP 2.0 带给我们最惊艳的莫过于多路复用了，虽然多路复用有种种好处，但是大家可以想一下，多路复用虽然好，但是它是建立在 TCP 连接的基础上，在连接频繁的情况下，是不是会对 TCP 连接造成压力，这个角度来讲，TCP 很容易成为性能瓶颈。

还有一点，使用 HTTP 2.0 会增加一次 TLS 握手过程，增加 RTT，这个我们上面也说到。

在 HTTP 2.0 中，多个请求是在同一个 TCP 管道中，这样当 HTTP 2.0 出现丢包时，整个 TCP 都要开始等待重传，那么就会阻塞该 TCP。连接中的所有请求。

2. HTTP 3.0

我们大家知道，HTTP 是应用层协议，应用层产生的数据会通过传输层协议作为载体来传输到互联网上的其他主机中，而其中的载体就是 TCP 协议，这是 HTTP 2 之前的主流模式。

但是随着 TCP 协议的缺点不断暴露出来，新一代的 HTTP 协议 – HTTP 3.0 毅然决然切断了和 TCP 的联系，转而拥抱了 UDP 协议，这么说不太准确，其实 HTTP 3.0 其实是拥抱了 **QUIC 协议**，而 QUIC 协议是建立在 UDP 协议基础上的。

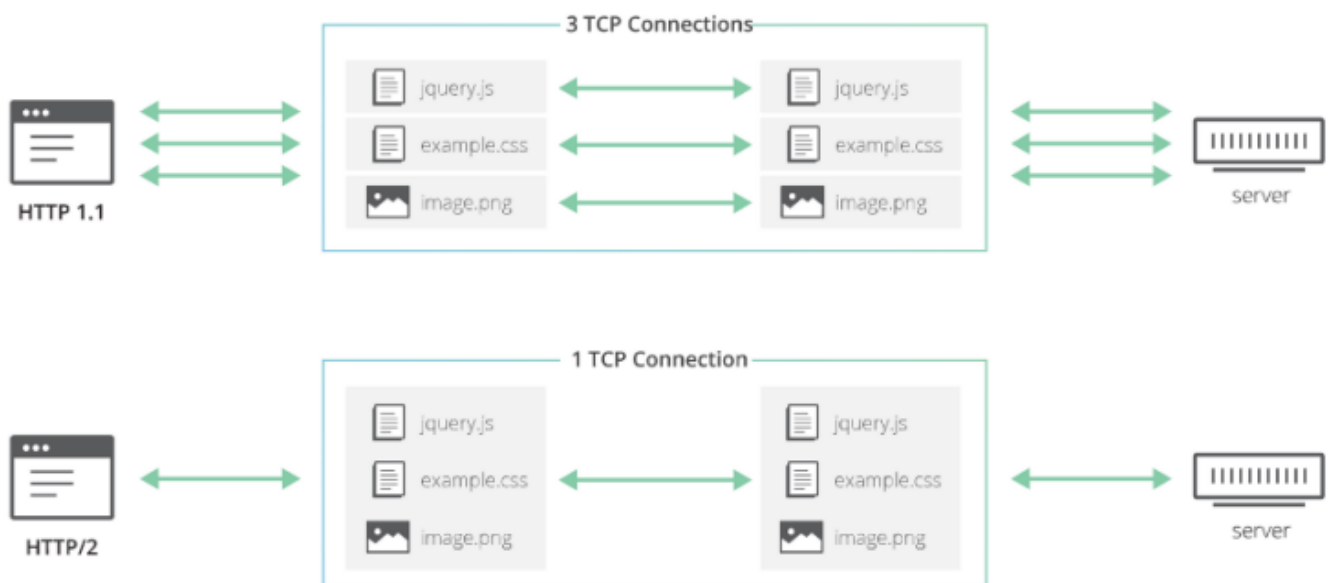
2.1. HTTP 3.0的诞生

HTTP 3.0 于 2022 年 6 月 6 日正式发布，IETF 把 HTTP 3.0 标准制定在了 RFC 9114 中，HTTP 3.0 其实相较于 HTTP 2.0 要比 HTTP 2.0 相较于 HTTP 1.1 的变化来说小很多，最大的提升就在于效率，替换 TCP 协议为 UDP 协议，HTTP 3.0 具有更低的延迟，它的效率甚至要比 HTTP 1.1 快 3 倍以上。

其实每一代 HTTP 协议的不断发展都是建立在上一代 HTTP 的缺点上的，就比如 HTTP 1.0 最大的问题就是传输安全性和不支持持久连接上，针对此出现了 HTTP 1.1，引入了 Keep-Alive 机制来保持长链接和 TLS 来保证通信安全性。但此时的 HTTP 协议并发性还做的不够好。

随着网络的不断发展，每个网站所需资源（CSS、JavaScript、图像等）的数量逐年增加，浏览器发现自己在获取和呈现网页时需要越来越多的并发性。但是由于 HTTP 1.1 只能够允许客户端/服务器进行一次 HTTP 请求交换，因此在网络层获得并发性的唯一方法是并行使用多个 TCP 连接到同一个源，不过使用多个 TCP 链接就失去了 keep-Alive 的意义。

然后出现了 SPDY 协议，主要解决 HTTP 1.1 效率不高的问题，包括降低延迟，压缩 header 等等，这些已经被 Chrome 浏览器证明能够产生优化效果，后来 HTTP 2.0 基于 SPDY，并且引入了 **流 (Stream)** 的概念，它允许将不同的 HTTP 交换多路复用到同一个 TCP 连接上，从而达到让浏览器重用 TCP 链接的目的。



TCP 的主要作用是以正确的顺序将整个字节流从一个端点传输到另一个端点，但是当流中的某些数据包丢失时，TCP 需要重新发送这些丢失的数据包，等到丢失的数据包到达对应端点时才能够被 HTTP

处理，这被称为 TCP 队头阻塞问题。

那么可能就会有人考虑到去修改 TCP 协议，其实这已经是一件不可能完成的任务了。因为 TCP 存在的时间实在太长，已经充斥在各种设备中，并且这个协议是由操作系统实现的，更新起来不大现实。

基于这个原因，Google 就更起炉灶搞了一个基于 UDP 协议的 QUIC 协议，并且使用在了 HTTP/3 上，HTTP/3 之前名为 HTTP-over-QUIC，从这个名字中我们也可以发现，HTTP/3 最大的改造就是使用了 QUIC。

